Modern selectors

Back in the selectors lesson, you learned the basics: element selectors, class selectors, ID selectors, groups, and descendants.

Modern CSS gives you more precise ways to ask questions about the HTML:

- Does this element have a certain attribute?
- Is this the current navigation link?
- Is this card marked as advanced?
- Is this item the last one in a list?
- Does this card contain a checked task?

That is the point of modern selectors. They help you use information that is already in the HTML, instead of adding a new class for every small styling case.

Use HTML that already has useful clues

In this lesson, you will work with a small lesson dashboard. The HTML has navigation links, lesson cards, difficulty levels, task checkboxes, and one external reference link. Put this markup in `index.html`:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Modern selector practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Use selectors that match real HTML clues</h1>
        <p class="summary">Modern selectors can match attributes, states, structure, and
      </section>

      <nav class="lesson-nav" aria-label="CSS lessons">
        <a href="/css/selectors">Selectors</a>
        <a href="/css/modern-selectors" aria-current="page">Modern selectors</a>
        <a href="https://developer.mozilla.org/">MDN reference</a>
      </nav>

      <section class="lesson-board" aria-label="Lesson cards">
        <article class="lesson-card" data-level="beginner">
          <p class="card-label">Beginner</p>
          <h2>Selectors recap</h2>
          <p>Review class selectors, group selectors, and descendant selectors.</p>
          <ul class="task-list">
```

```
    <li><label><input type="checkbox" checked> Read the lesson</label></li>
    <li><label><input type="checkbox"> Try the exercises</label></li>
  </ul>
</article>

<article class="lesson-card" data-level="advanced">
  <p class="card-label">Advanced</p>
  <h2>Modern selectors</h2>
  <p>Use attributes, structure, and parent-aware selectors carefully.</p>
  <p class="card-note">Best after you understand specificity.</p>
  <ul class="task-list">
    <li><label><input type="checkbox"> Read the lesson</label></li>
    <li><label><input type="checkbox"> Inspect the matches</label></li>
  </ul>
</article>

<article class="lesson-card" data-level="beginner">
  <p class="card-label">Beginner</p>
  <h2>Practice project</h2>
  <p>Apply selectors to a small lesson dashboard.</p>
  <ul class="task-list">
    <li><label><input type="checkbox" checked> Build the layout</label></li>
    <li><label><input type="checkbox" checked> Style the cards</label></li>
  </ul>
</article>
      </section>
    </main>
  </body>
</html>
```

Notice the useful clues already in the HTML:

- The current nav link has `aria-current="page"`.
- The external link starts with `https://`.
- Cards have `data-level="beginner"` or `data-level="advanced"`.

Previous Lesson Next Lesson

Modern selectors let CSS use those clues directly.

Add a simple base first

Start with the base styling so the page is readable before the selector-specific rules appear:

css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f3f4f6;
}

.page {
  max-width: 1040px;
  margin: 40px auto;
  padding: 0 16px;
}

.intro,
.lesson-nav,
```

```
.lesson-card {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
}

.intro,
.lesson-nav,
.lesson-card {
  padding: 24px;
}

.intro,
.lesson-nav {
  margin-bottom: 24px;
}

.eyebrow,
.card-label {
  margin-top: 0;
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
h2,
p {
  margin-top: 0;
}

p,
.task-list {
  color: #4b5563;
  line-height: 1.6;
}

.lesson-nav {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 12px;
}

.lesson-nav a {
  color: #2563eb;
  font-weight: 700;
  text-underline-offset: 3px;
}

.lesson-board {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}

.task-list {
  padding-left: 20px;
  margin-bottom: 0;
}
```

This CSS does not use modern selectors yet. It only gives the page a normal layout and readable card styles.

The next sections will use more precise selectors to style the clues already present in the HTML.

Select attributes when the HTML already says something useful

An attribute selector matches an element by one of its HTML attributes.

The current nav link already says `aria-current="page"`. You do not need to add a class like `.active-link` just to style it. Add this CSS to make the active link visible:

```css
.lesson-nav a[aria-current="page"] {
  color: white;
  background-color: #2563eb;
  border-radius: 999px;
  padding: 6px 10px;
  text-decoration: none;
}
```

`a[aria-current="page"]` means "match an `<a>` element whose `aria-current` attribute is exactly `page`."

The selector is scoped under `.lesson-nav` so it only styles the current link in this navigation area.

The CSS changes the current link's color, background, shape, spacing, and underline. The meaning still comes from the HTML attribute. CSS is only making that state visible.

Now style the advanced card using its data attribute:

```css
.lesson-card[data-level="advanced"] {
  border-color: #f59e0b;
  background-color: #fffbeb;
}
```

`[data-level="advanced"]` matches a card with that exact custom data attribute.

Data attributes are useful when the HTML carries app-specific information that CSS can safely style. Here, `data-level` tells CSS whether a card is beginner or advanced.

> **Prefer meaningful existing hooks** - If the HTML already has a meaningful attribute like `aria-current`, `disabled`, `required`, or a clear `data-*` value, an attribute selector can be better than adding another styling-only class.

Match part of an attribute value

Some attribute selectors can match part of a value. The MDN link starts with `https://`, which tells you it goes to a full external URL.

Add a selector for links that start with `https://`:

```css
.lesson-nav a[href^="https://"] {
  color: #7c3aed;
}
```

`[href^="https://"]` means "match links whose `href` value starts with `https://`."

The `^=` operator means "starts with." This kind of selector is useful for styling external links, file links, email links, or download links when the attribute value already reveals the type.

Keep the styling subtle here. The link should still look like a normal link, and the color only adds a small distinction.

Select structure instead of adding edge-case classes

Sometimes the useful clue is not an attribute. It is the element's position in a group.

The task list has several list items. If you want space between items, you could add a class to every item after the first one, but the structure already tells CSS what to do.

Add spacing between list items with the adjacent sibling selector:

```css
.task-list li + li {
  margin-top: 8px;
}
```

`li + li` means "match a list item that comes immediately after another list item." The first item does not match, because it does not come after a previous item. The later items match, so they get top margin.

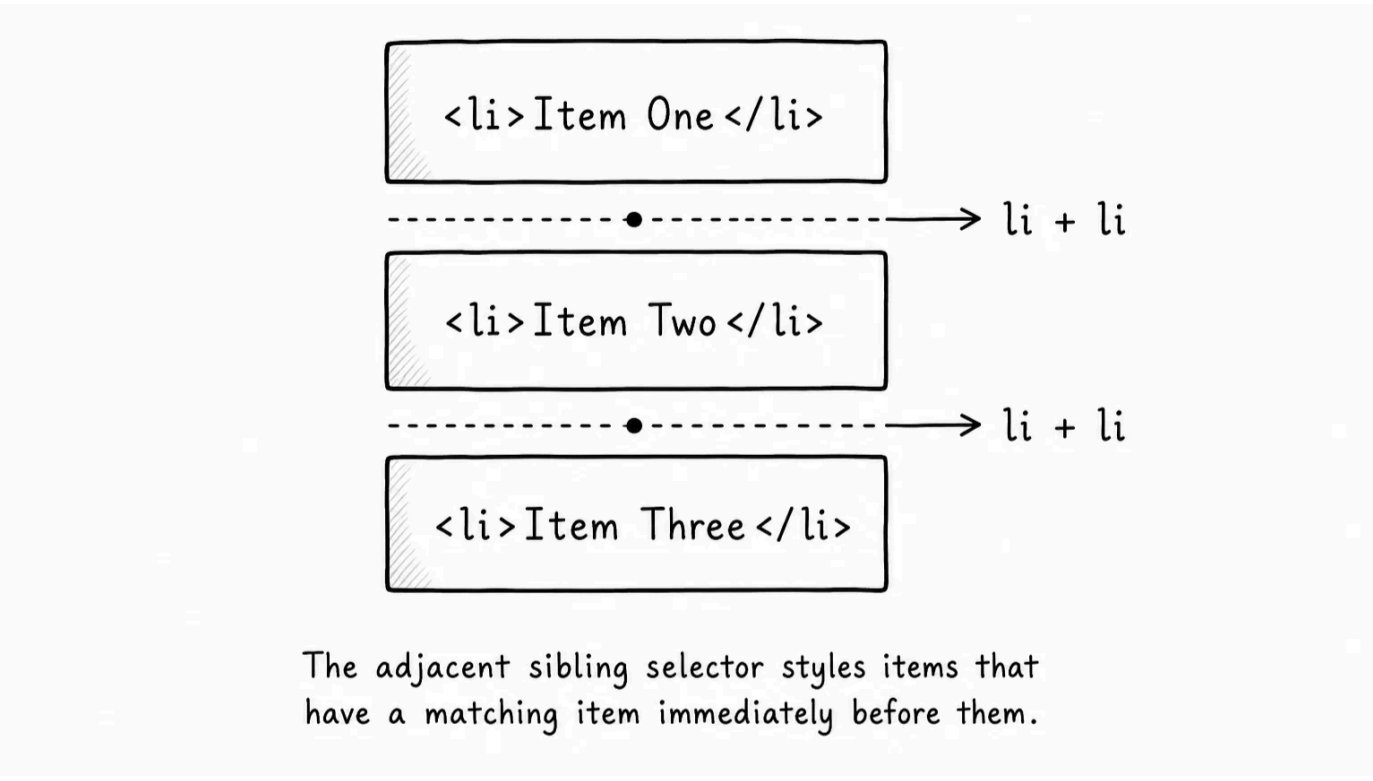
This is useful for spacing repeated items because it only creates space between items, not before the first item.

You can also select the last item in a group:

```css
.task-list li:last-child {
  font-weight: 700;
}
```

`:last-child` matches an element when it is the last child inside its parent.

This example makes the last task heavier so you can see the match. In real projects, structural selectors are often used for spacing, borders, dividers, and small layout adjustments.



Use `:not()` to exclude a case

Sometimes the useful question is negative: "match cards except this kind." For example, you may want a quieter border on every card that is not advanced:

```css
.lesson-card:not([data-level="advanced"]) {
  border-color: #dbeafe;
}
```

`:not(...)` means "match this element only if it does not match the selector inside the parentheses."

Here, `.lesson-card:not([data-level="advanced"])` matches lesson cards that are not advanced.

Use this when the exception is clear. Do not build long negative selectors that become hard to read. A simple class is often better when the logic gets complicated.

Use `:is()` to group selector options

Sometimes several selectors should receive the same style. You already know comma grouping:

```css
.lesson-card h2,
.lesson-card p,
.lesson-card ul {
  margin-bottom: 12px;
```

```
}
```

`:is()` gives you another way to write that group:

```css
.lesson-card :is(h2, p, ul) {
  margin-bottom: 12px;
}
```

`:is(h2, p, ul)` means "match any of these options."

The full selector means "inside `.lesson-card`, match any `h2`, `p`, or `ul`."

This is useful when the shared part of the selector is long. Instead of repeating `.lesson-card` three times, you write it once.

Use `:is()` for readability, not to make selectors look clever.

Use `:where()` for low-specificity defaults

`:where()` looks similar to `:is()`, but it has a different specificity behavior.

Specificity is selector strength, and you learned earlier that stronger selectors can override weaker selectors.

`:where()` matches elements like `:is()`, but the part inside `:where()` adds no specificity. That makes it useful for broad defaults you want to keep easy to override:

```css
.lesson-card :where(h2, p, ul) {
  margin-top: 0;
}
```

This sets simple spacing defaults inside cards without making the selector stronger than it needs to be.

Later, a normal class like `.card-note` can override those defaults easily:

```css
.card-note {
  margin-top: 16px;
  color: #92400e;
  font-weight: 700;
}
```

`.card-note` can set its own margin because the default came from `:where()` instead of a stronger selector.

The practical habit is simple: use `:where()` when you are writing gentle defaults. Use normal classes when you are styling a specific component part.

> ⚠ **Do not fight yourself with overly strong selectors** - Modern selectors can become very specific if you chain too much together. If you need three ancestors, two attributes, and a state to style something, ask whether a clear class would be easier.

Use `:has()` when the parent depends on its contents

Most selectors match an element based on itself or its ancestors.

`:has()` lets you match a parent based on something inside it.

In the lesson cards, some tasks are already checked. You can style a card differently when it contains at least one checked task:

```css
.lesson-card:has(.task-list input:checked) {
  box-shadow: 0 12px 24px rgba(15, 23, 42, 0.08);
}
```

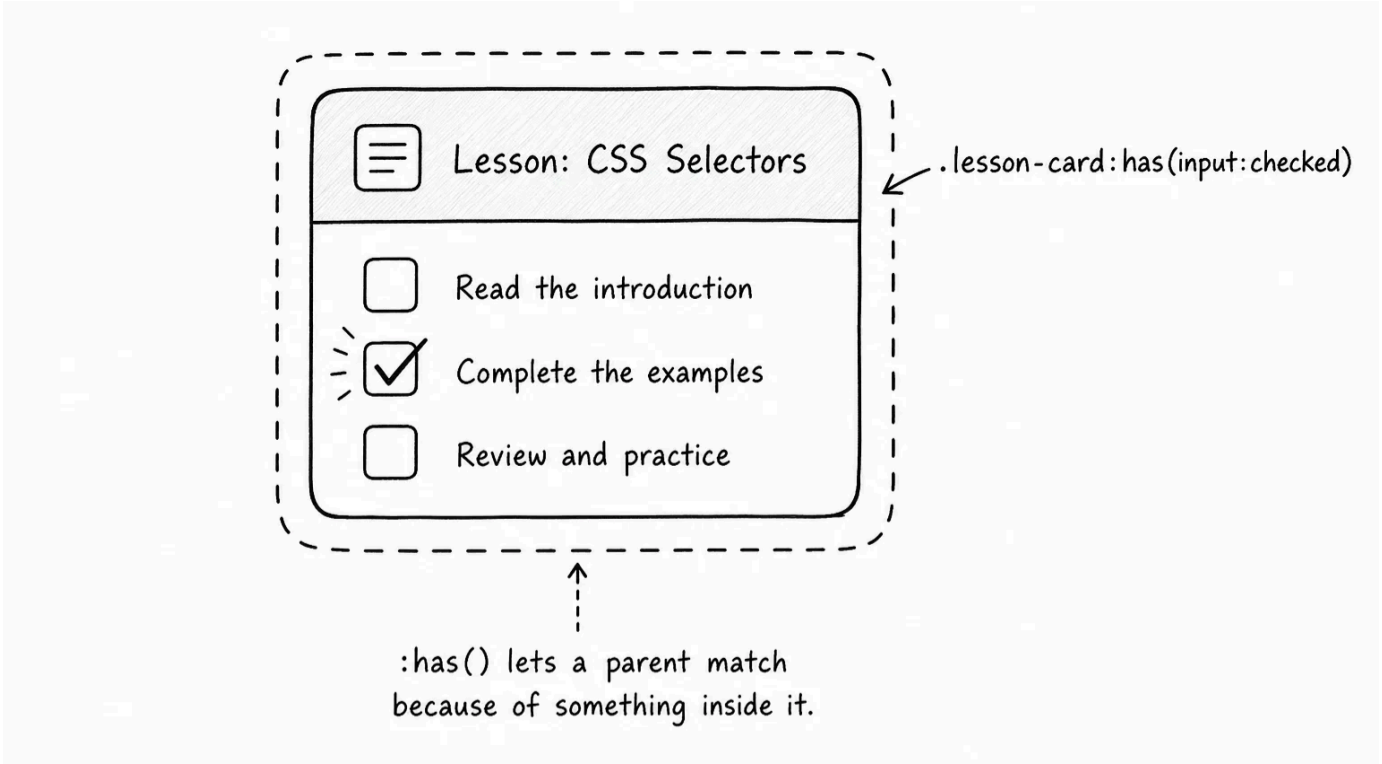
`:has(.task-list input:checked)` means "match this card if it has a checked input inside `.task-list`."

The selector reads like a condition:

```text
Select a lesson card if it has a checked task.
```

That is useful for parent styling. The card can react to something inside it without adding a separate `.has-progress` class.

Use `:has()` when the parent truly depends on a child or descendant. If the parent already has a meaningful class or data attribute, that simpler hook may still be better.



Inspect which selectors match

DevTools is still the best way to check selector behavior.

Inspect the current nav link. You should see the `.lesson-nav a[aria-current="page"]` rule in the Styles panel.

Inspect the MDN link. You should see the `a[href^="https://"]` rule.

Inspect the advanced card. You should see the `[data-level="advanced"]` rule.

Inspect a card with checked tasks. You should see the `:has()` rule if the card contains a checked input.

If a selector does not work, check the exact HTML:

- Does the attribute exist?
- Does the attribute value match exactly?
- Is the element in the structure your selector expects?
- Is the checked state actually present?
- Is another rule winning in the cascade?

Modern selectors are powerful, but they are still selectors. They only match what the browser can actually see in the DOM.

Modern selector map

Here is the practical map:

Selector	Question it asks
`[aria-current="page"]`	Does this element have this exact attribute value?

Selector	Question it asks
`[href^="https://"]`	Does this attribute value start with this text?
`li + li`	Does this element come immediately after another matching
`:last-child`	Is this element the last child of its parent?
`:not(...)`	Does this element not match this condition?
`:is(...)`	Does this element match any option in this group?
`:where(...)`	Does this element match any option, without adding selector
`:has(...)`	Does this element contain something that matches?

The useful habit is not to use the newest selector everywhere. The useful habit is to use what is clearer and makes sense.

Try it

Use the same dashboard and test the selectors directly:

- Change `aria-current="page"` to another nav link. The current-link style should move.
- Change the MDN link from `https://developer.mozilla.org/` to `/docs`. The external-link color should stop applying.
- Change a card from `data-level="advanced"` to `data-level="beginner"`. Notice which card styles change.
- Add a new task to the bottom of a task list. Check how `li + li` and `:last-child` behave.
- Remove every checked checkbox from one card. The `:has()` shadow should disappear from that card.
- Replace the `:is()` rule with the comma-grouped version and confirm the result is the same.
- Try overriding the `:where()` margin rule with a simple class. Notice that the class can win easily.

Modern selectors are best when they make your CSS match the real meaning or structure already in the HTML. If a selector becomes hard to explain out loud, it is probably too clever.

0 / 10

Knew **0 Items** Learnt **0 Items** Skipped **0 Items** ResetProgress

Test Your Understanding

What problem do modern selectors solve?

Click to Reveal the Answer

Already Know that | Didn't Know that | Skip Question

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →